HDOC Day-46

Bubble Sort and Finding First and Last Occurrences in C

Aim

To sort a variable-length integer array in ascending order using Bubble_sort() and identify the first and last zero-based indices of a target using occurrence(), passing arrays through pointers.

Algorithm

1. Read the array size n and validate that n is positive.
2. Declare nums[n] and read n integers.
3. Pass nums and n to Bubble_sort().
4. Use nested loops to compare adjacent elements and swap them into ascending order.
5. Display the sorted array.
6. Call occurrence() and read the target integer.
7. Initialize first = -1 and last = -1; scan the sorted array.
8. Record the first matching index and update the last index for every match.
9. Display the target values and indices, or -1, -1 when absent.
10. Stop the program.

C Program

```
#include <stdio.h>

void Bubble_sort(int *nums, int n);
void occurrence(int *nums, int n);

int main()
{
    int n, i;
    printf("Enter the size of array: ");
    scanf("%d", &n);
    if (n <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }
    int nums[n];
    printf("Enter %d array elements: ", n);
    for (i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    Bubble_sort(nums, n);
    printf("\nSorted Array: ");
    for (i = 0; i < n; i++)
        printf("%d ", nums[i]);
    printf("\n");
    occurrence(nums, n);
    return 0;
}
```

```
void Bubble_sort(int *nums, int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            if (*(nums + j) > *(nums + j + 1))
            {
                temp = *(nums + j);
                *(nums + j) = *(nums + j + 1);
                *(nums + j + 1) = temp;
            }
        }
    }
}

void occurrence(int *nums, int n)
{
    int target, first = -1, last = -1, i;
    printf("Enter target element: ");
    scanf("%d", &target);
    for (i = 0; i < n; i++)
    {
        if (*(nums + i) == target)
        {
            if (first == -1)
                first = i;
            last = i;
        }
    }
    if (first == -1)
    {
        printf("Target not found.\n");
        printf("First and Last Indices: -1, -1\n");
    }
    else
    {
        printf("First Occurrence:\n");
        printf("Value: %d, Index: %d\n", *(nums + first), first);
        printf("Last Occurrence:\n");
        printf("Value: %d, Index: %d\n", *(nums + last), last);
        printf("First and Last Indices: %d, %d\n", first, last);
    }
}
```

Test Case Table

Case	Input Array	Target	First, Last Index
1	5 2 8 2 9 2	2	(0, 2)
2	10 20 30 40 50	30	(2, 2)
3	4 4 4 4 4	4	(0, 4)
4	1 3 5 7 9	6	(-1, -1)
5	8 3 1 8 5 8 2	8	(4, 6)
6	15 10 20 10 5	10	(1, 2)

Case	Input Array	Target	First, Last Index
7	7	7	(0, 0)
8	7	3	(-1, -1)
9	n = 0	-	Invalid array size

Sample Output

```
Enter the size of array: 7
Enter 7 array elements: 8 3 1 8 5 8 2

Sorted Array: 1 2 3 5 8 8 8
Enter target element: 8

First Occurrence:
Value: 8, Index: 4
Last Occurrence:
Value: 8, Index: 6
First and Last Indices: 4, 6
```

Sample Output - Target Not Found

```
Enter the size of array: 5
Enter 5 array elements: 10 20 30 40 50

Sorted Array: 10 20 30 40 50
Enter target element: 25
Target not found.
First and Last Indices: -1, -1
```

Result

The C program was designed using variable-length arrays, pointers, user-defined functions, and Bubble Sort. It sorts elements in ascending order, reports the values and zero-based indices of the first and last occurrences of a target, and displays -1, -1 when the target is absent.